Smart Music Playlist Optimizer

Authors: Kirill Ryrmak, Heigo Tornik, Erik Presnov

University of Tartu, Institute of Computer Science, Algorithmics (MTAT.03.238)

ABSTRACT

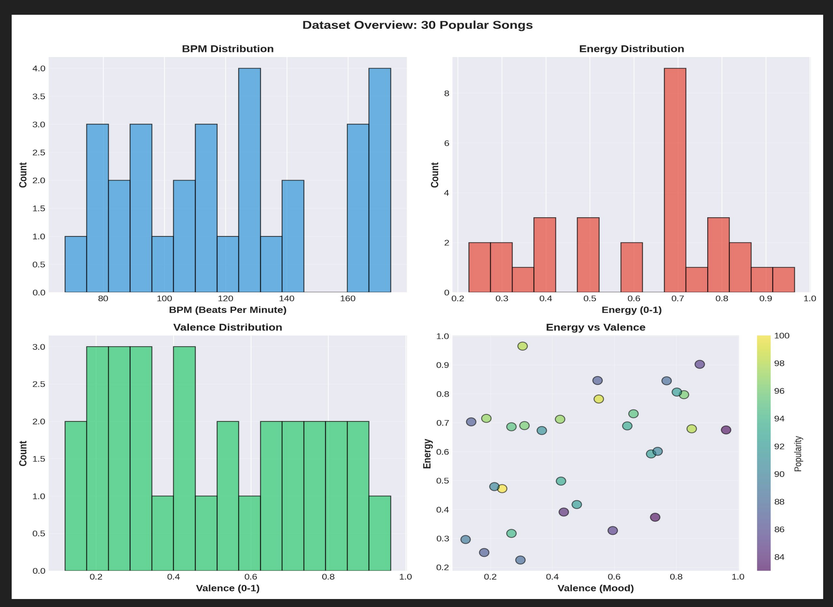
Random shuffle creates jarring transitions in music playlists by placing songs with incompatible tempos, energy levels, and moods next to each other. We developed a graph-based optimization system that models playlist ordering as a Traveling Salesman Problem variant, where songs are nodes and transition costs are edge weights.

INTRODUCTION

Music streaming platforms commonly use random shuffle for playlists, which creates harsh transitions between songs with different audio characteristics. A 140 BPM rock song followed by a 60 BPM ballad disrupts listening flow and reduces user enjoyment. This project addresses the playlist optimization problem by automatically reordering songs to minimize perceptual discontinuity.

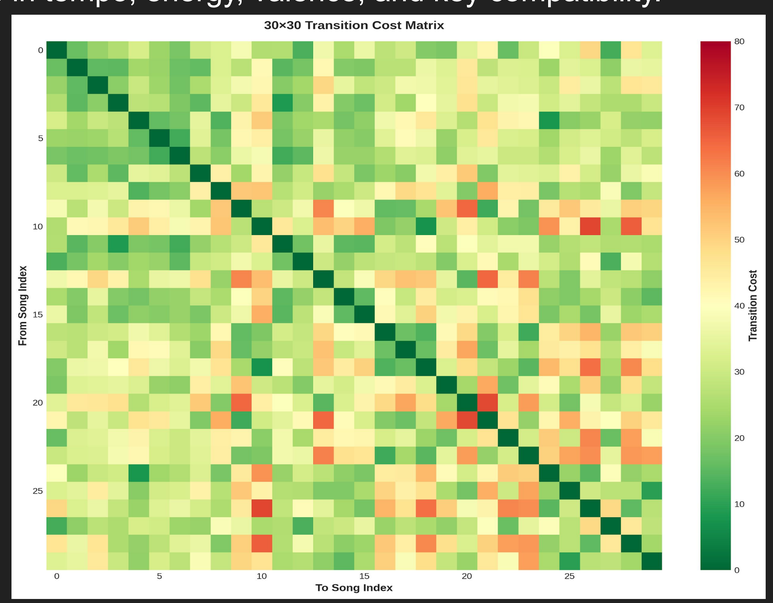
We model this as a weighted graph where each song is a node and edge weights represent transition costs based on audio features. The optimization goal is finding a song ordering that minimizes total transition cost while maintaining musical diversity. Audio features from Spotify API include tempo (BPM), energy, valence (mood), and musical key. Our cost function weights these features as: 0.4×tempo + 0.3×energy + 0.2×valence + 0.1×key_penalty, prioritizing tempo smoothness as most perceptually important. The target was achieving 50-60% improvement over random baseline.

Dataset: We highlight an example of 30 popular songs from Spotify dataset with popularity scores above 83 for the detailed analysis. The selection ensures diversity with BPM ranging from 67.5 to 173.9, energy from 0.23 to 0.96, and valence from 0.12 to 0.96. This wide range creates a challenging optimization problem with 870 possible transitions between song pairs. For general algorithm performance we benchmarked playlist sizes starting from 5 up to 1000.



IMPLEMENTATION

Our pipeline begins with filtering 114,000 songs from Spotify's dataset to select 30 diverse tracks based on popularity and feature distribution. We construct a 30×30 cost matrix containing all 870 pairwise transition costs, precomputed for O(1) lookup during optimization. The cost function combines normalized differences in tempo, energy, valence, and key compatibility.

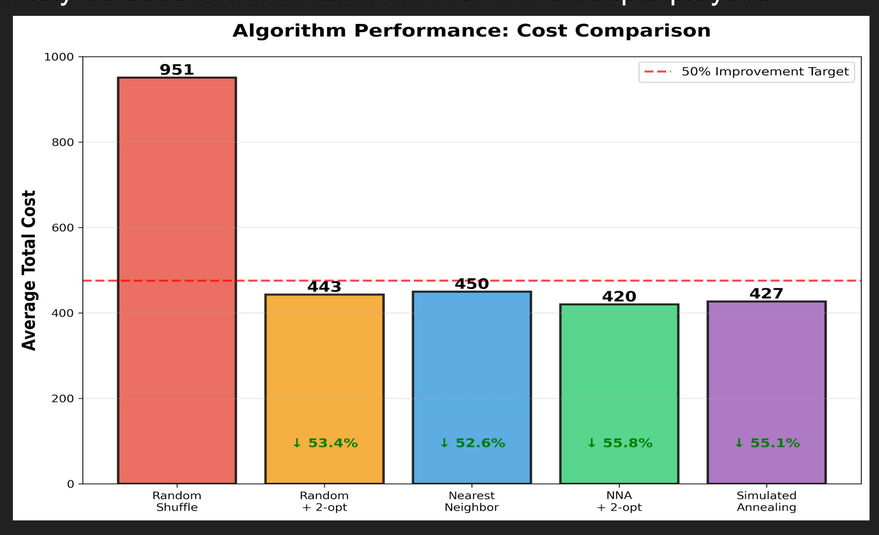


Nearest Neighbor Algorithm constructs an initial solution by starting from a random song and iteratively selecting the nearest unvisited song based on minimum transition cost. This greedy approach runs in O(n²) time and for our example achieved a 450 average cost, representing 52% improvement over random ordering. NNA provides a strong baseline but can get trapped in local optima.

2-opt Local Search refines any initial solution through iterative edge swapping. For each pair of edges, it tests whether reversing the segment between them reduces total cost. The algorithm runs in O(n²×k) time where k represents iterations until convergence, typically 5-15 for our dataset. Applied to our example, 2-opt achieves 443 average cost (53% improvement).

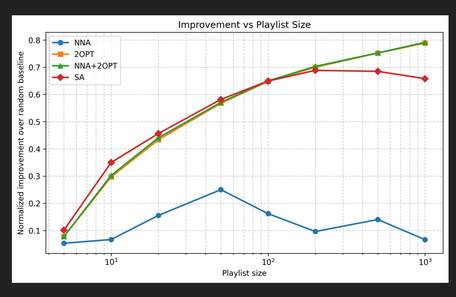
When combined with NNA output, it reaches 420 average cost (56% improvement) in 1.5 milliseconds.

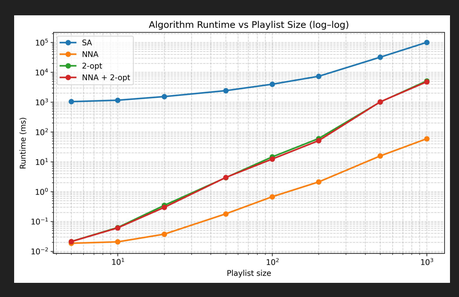
Simulated Annealing was implemented for comparison as a global optimization approach. SA uses temperature-based acceptance probability exp(-Δcost/T) with cooling schedule T×0.9999 per iteration. The neighbor generation combines 60% 2-opt moves, 25% random swaps, and 15% double-bridge escapes. After 200,000 iterations across 20 runs, SA achieved 427 average cost (55% improvement) but required approximately 60 seconds execution time for our example playlist.

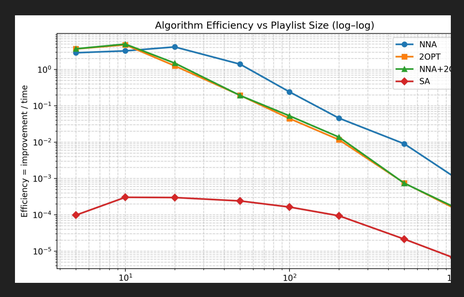


RESULTS

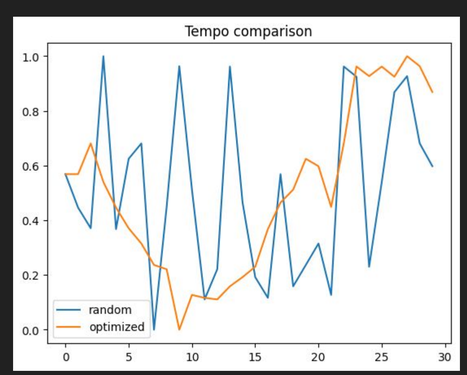
Benchmarking the algorithms on playlist sizes ranging from 5 - 1000 showed, that for smaller playlists the random shufflings have a higher possibility of being closer to the global minimum, therefore algorithm cost reduction generally improves with larger playlists. NNA while fast does not improve the baseline as well for large playlist like 2-opt (+NNA) and SA does. Comparing general efficiency (cost reduced vs time needed) NNA is the best followed by 2-opt (+NNA) with SA the least efficient.

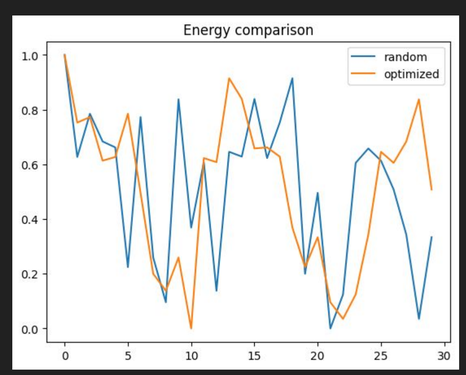


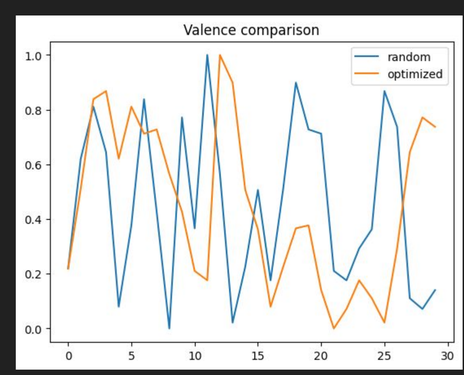


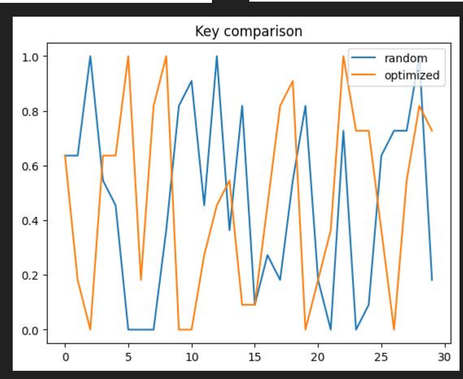


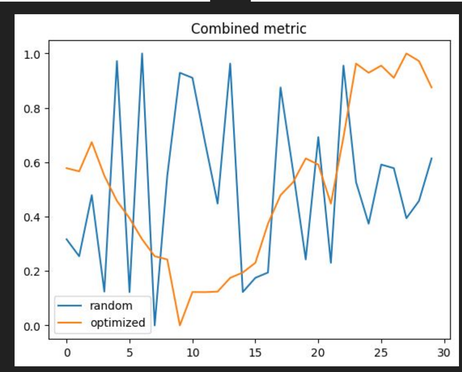
Comparisons of different components of the cost function. Tempo, which has the highest weight, gets the best optimization and is quite smooth compared to random ordering. Energy is also notably better than random. Valence and key have quite small weights in the cost function so they do not get as optimized and thus are still a bit jumpy. The last graph shows that overall, the transitions are quite a bit smoother than random.



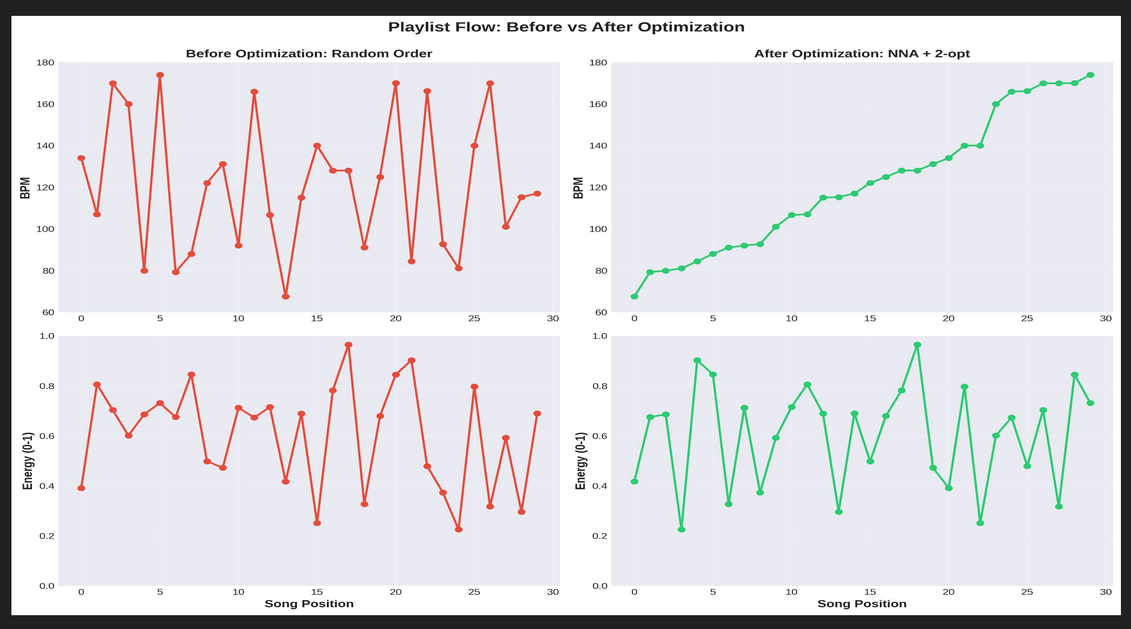








The before-after comparison clearly illustrates transformation from chaotic random ordering with frequent jarring jumps to systematic progression that maintains musical diversity while minimizing perceptual discontinuity between consecutive songs.



CONCLUSIONS

Testing on 30 diverse Spotify songs showed 56% reduction in transition cost compared to random ordering, exceeding our 50-60% target goal. Generally 2-opt (+NNA) provides the best balance of speed and cost reduction across the board for different playlist sizes. NNA is fast, but its improvement over a random baseline with large playlists plateaus. SA is too slow and inefficient for practical use.